



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



TECNOLÓGICO
NACIONAL DE MÉXICO



Instituto Tecnológico de Tijuana

Ingeniería en Sistemas Computacionales

Unidad 3

Práctica : Diseño de Estado

Maestra:

Maribel Guerrero Luis

Alumnos:

Esmeralda Gamboa Hernandez - 21210374

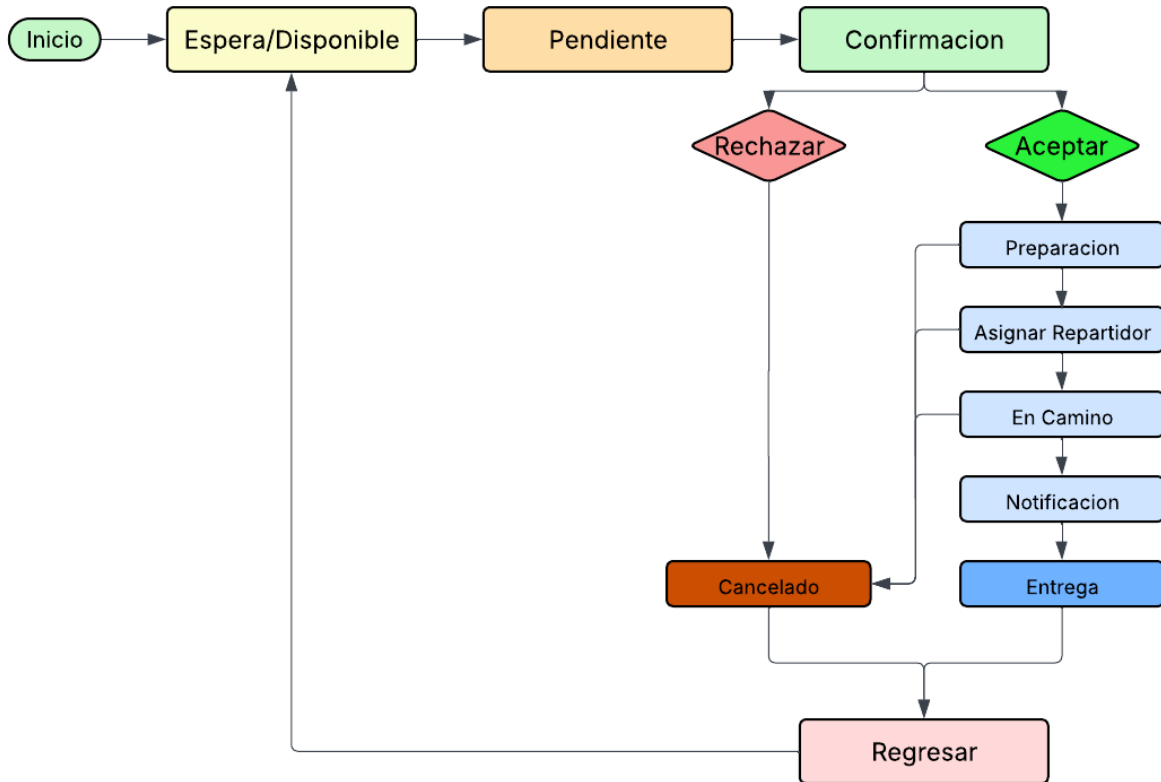
3:00 p.m. – 4:00 p.m.

5 de Mayo del 2026

Diagrama de Estados

Uber Eats

Patron: Diseño de Estado



Código

```
using System;  
using System.Drawing;  
using System.Windows.Forms;  
  
namespace UberEatsApp  
{
```

```
// ===== FORM =====
```

```
public partial class Form1 : Form
```

```
{
```

```
    private Pedido pedido;
```

```
    private bool notificacionMostrada = false;
```

```
    public Form1()
```

```
    {
```

```
        InitializeComponent();
```

```
        pedido = new Pedido(new EstadoEspera());
```

```
        ActualizarUI("Sistema iniciado");
```

```
    }
```

```
    private void ActualizarUI(string accion)
```

```
    {
```

```
        string estado = pedido.Estado.GetType().Name.Replace("Estado", "");
```

```
        lblEstado.Text = $"Estado: {estado}";
```

```
        listHistorial.Items.Add(
```

```
            $"[{DateTime.Now:T}] {accion} → {estado}"
```

```
        );
```

```
// Colores por estado

if (pedido.Estado is EstadoEspera)

    lblEstado.BackColor = Color.LightGreen;
else if (pedido.Estado is EstadoPendiente)

    lblEstado.BackColor = Color.Khaki;
else if (pedido.Estado is EstadoConfirmacion)

    lblEstado.BackColor = Color.LightSeaGreen;
else if (pedido.Estado is EstadoPreparacion)

    lblEstado.BackColor = Color.MediumPurple;
else if (pedido.Estado is EstadoAsignacion)

    lblEstado.BackColor = Color.SkyBlue;
else if (pedido.Estado is EstadoEnCamino)

    lblEstado.BackColor = Color.CadetBlue;
else if (pedido.Estado is EstadoNotificacion)

    lblEstado.BackColor = Color.Orange;
else if (pedido.Estado is EstadoEntrega)

    lblEstado.BackColor = Color.LightBlue;
else if (pedido.Estado is EstadoCancelado)

    lblEstado.BackColor = Color.IndianRed;


ActualizarBarra();

if (pedido.Estado is EstadoNotificacion && !notificacionMostrada)
{
    notificacionMostrada = true;
```

```

        FormNotificacion frm = new FormNotificacion();
        frm.ShowDialog();
    }

    // Resetear cuando no esté en notificación
    if (!(pedido.Estado is EstadoNotificacion))
    {
        notificacionMostrada = false;
    }
}

private void btnSiguiente_Click_1(object sender, EventArgs e)
{
    pedido.Siguiente();
    ActualizarUI("Avanzar");
}

private void btnCancelar_Click_1(object sender, EventArgs e)
{
    pedido.Cancelar();
    ActualizarUI("Cancelar");
}

private void btnRegresar_Click_1(object sender, EventArgs e)
{

```

```
pedido.Estado = new EstadoEspera();  
ActualizarUI("Regresar");  
}
```

```
private void ActualizarBarra()  
{  
    Label[] pasos = {  
        lblPaso1, lblPaso2, lblPaso3, lblPaso4,  
        lblPaso5, lblPaso6, lblPaso7  
    };  
};
```

```
// Resetear colores  
foreach (var paso in pasos)  
{  
    paso.BackColor = Color.Gray;  
    paso.ForeColor = Color.White;  
}
```

```
int indice = 0;
```

```
if (pedido.Estado is EstadoEspera) indice = 0;  
else if (pedido.Estado is EstadoPendiente) indice = 1;  
else if (pedido.Estado is EstadoConfirmacion) indice = 2;  
else if (pedido.Estado is EstadoPreparacion) indice = 3;  
else if (pedido.Estado is EstadoAsignacion) indice = 4;
```

```
else if (pedido.Estado is EstadoEnCamino) indice = 5;  
else if (pedido.Estado is EstadoNotificacion) indice = 5;  
else if (pedido.Estado is EstadoEntrega) indice = 6;
```

```
// Pintar progreso
```

```
for (int i = 0; i <= indice; i++)
```

```
{  
    pasos[i].BackColor = Color.LimeGreen;  
}
```

```
// Si está cancelado
```

```
if (pedido.Estado is EstadoCancelado)
```

```
{  
    foreach (var paso in pasos)  
    {  
        paso.BackColor = Color.IndianRed;  
    }  
}
```

```
private void pictureBox1_Click(object sender, EventArgs e)
```

```
{
```

```
}
```

```
}
```

```
// ===== STATE =====
```

```
public interface IEstadoPedido  
{  
    void Siguiente(Pedido contexto);  
    void Cancelar(Pedido contexto);  
}
```

```
// ===== CONTEXTO =====
```

```
public class Pedido  
{  
    public IEstadoPedido Estado { get; set; }  
  
    public Pedido(IEstadoPedido estadoInicial)  
    {  
        Estado = estadoInicial;  
    }  
  
    public void Siguiente() => Estado.Siguiente(this);  
    public void Cancelar() => Estado.Cancelar(this);  
}
```

```
// ===== ESTADOS =====
```

```
public class EstadoEspera : IEstadoPedido  
{  
    public void Siguiente(Pedido c) => c.Estado = new EstadoPendiente();  
}
```



```
public void Cancelar(Pedido c) {}  
}
```

```
public class EstadoPendiente : IEstadoPedido  
{  
    public void Siguiente(Pedido c) => c.Estado = new EstadoConfirmacion();  
    public void Cancelar(Pedido c) => c.Estado = new EstadoCancelado();  
}
```

```
public class EstadoConfirmacion : IEstadoPedido  
{  
    public void Siguiente(Pedido c) => c.Estado = new EstadoPreparacion();  
    public void Cancelar(Pedido c) => c.Estado = new EstadoCancelado();  
}
```

```
public class EstadoPreparacion : IEstadoPedido  
{  
    public void Siguiente(Pedido c) => c.Estado = new EstadoAsignacion();  
    public void Cancelar(Pedido c) => c.Estado = new EstadoCancelado();  
}
```

```
public class EstadoAsignacion : IEstadoPedido  
{  
    public void Siguiente(Pedido c) => c.Estado = new EstadoEnCamino();  
    public void Cancelar(Pedido c) => c.Estado = new EstadoCancelado();  
}
```

```
}
```

```
public class EstadoEnCamino : IEstadoPedido
```

```
{
```

```
    public void Siguiente(Pedido c) => c.Estado = new EstadoNotificacion();
```

```
    public void Cancelar(Pedido c) => c.Estado = new EstadoCancelado();
```

```
}
```

```
public class EstadoNotificacion : IEstadoPedido
```

```
{
```

```
    public void Siguiente(Pedido c) => c.Estado = new EstadoEntrega();
```

```
    public void Cancelar(Pedido c) { }
```

```
}
```

```
public class EstadoEntrega : IEstadoPedido
```

```
{
```

```
    public void Siguiente(Pedido c) => c.Estado = new EstadoEspera();
```

```
    public void Cancelar(Pedido c) { }
```

```
}
```

```
public class EstadoCancelado : IEstadoPedido
```

```
{
```

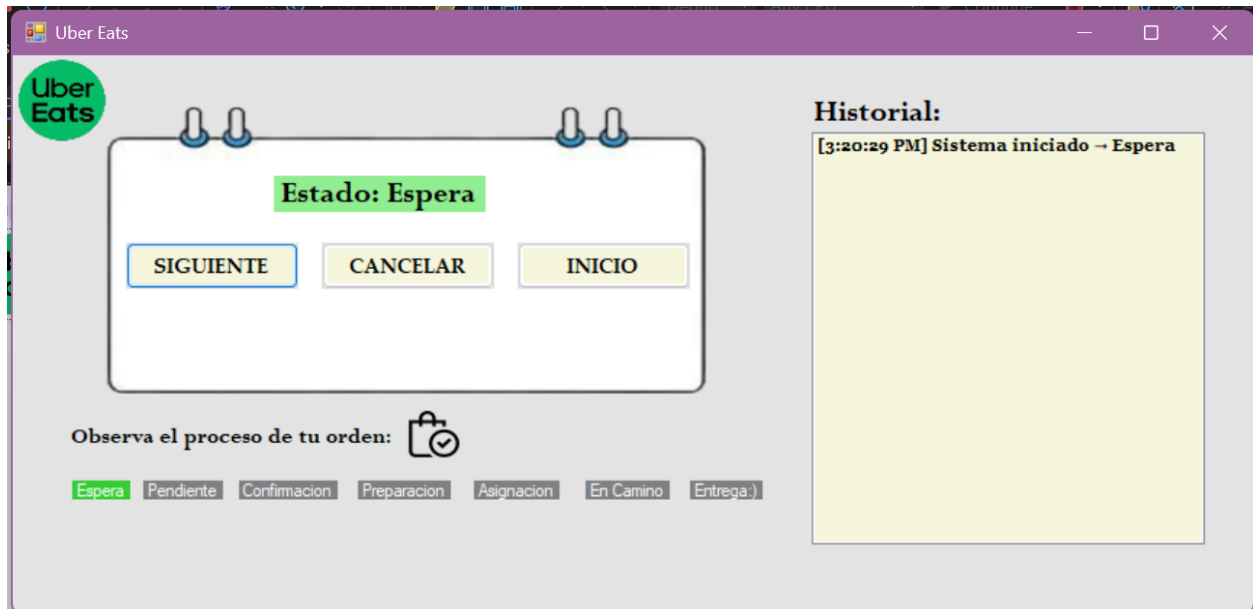
```
    public void Siguiente(Pedido c) => c.Estado = new EstadoEspera();
```

```
    public void Cancelar(Pedido c) { }
```

```
}
```

}

Pantallas



Conclusión

Me agrado el realizar esta práctica, fue un patrón sencillo de entender y me resulto sencillo el realizar el problema requerido, así fuera de manera individual.